

Implementation and Verification of a CAN-FD Bus Transceiver in VHDL

Mads Richardt (s224948)

February 28, 2026

Contents

1	Design and Architecture	1
1.1	System Overview	1
1.2	LLC Frame Format	1
1.3	Interface Definition Tables	4
1.4	Protocol-Driven Type System	11
1.5	LLC Sub-layer	12
1.5.1	can_llc_tx	12
1.5.2	can_llc_rx	12
1.6	MAC Sub-layer	12
1.6.1	can_mac_tx	12
1.6.1.1	can_mac_tx Internal Interfaces	12
1.6.1.2	can_mac_fsm_tx	15
1.6.1.3	can_mac_ser_tx	16
1.6.1.4	can_mac_bs_tx	16
1.6.1.5	can_mac_crc_tx	16
1.6.2	can_mac_rx	16
1.7	PCS Sub-layer	16
1.7.1	can_pcs_tx	16
1.7.2	can_pcs_rx	19

1 Design and Architecture

1.1 System Overview

A complete CAN node decomposes into a TX path and an RX path, coordinated by shared Fault Confinement Entity (FCE) and Physical Medium Attachment (PMA) control, as shown in Figure 1. Each path spans three sub-layers - LLC (Section 1.5), MAC (Section 1.6), and PCS (Section 1.7) - with the LLC frame format defined in Section 1.2 and interface bundles defined in Section 1.3. A protocol-driven type system (Section 1.4) underpins all inter-module interfaces, encoding ISO semantics directly into the VHDL type hierarchy.

1.2 LLC Frame Format

The LLC Frame format used by the existing CAN-bus implementation (can_bus_controller) is depicted in Figure 2 with the ID byte format depicted in Table 1. A revised LLC Frame supporting FD is depicted in Figure 3. The revised format adds a 3-bit FMT [1, Sec. 6.4.3] field to the DLC byte (LLC Frame byte 4),

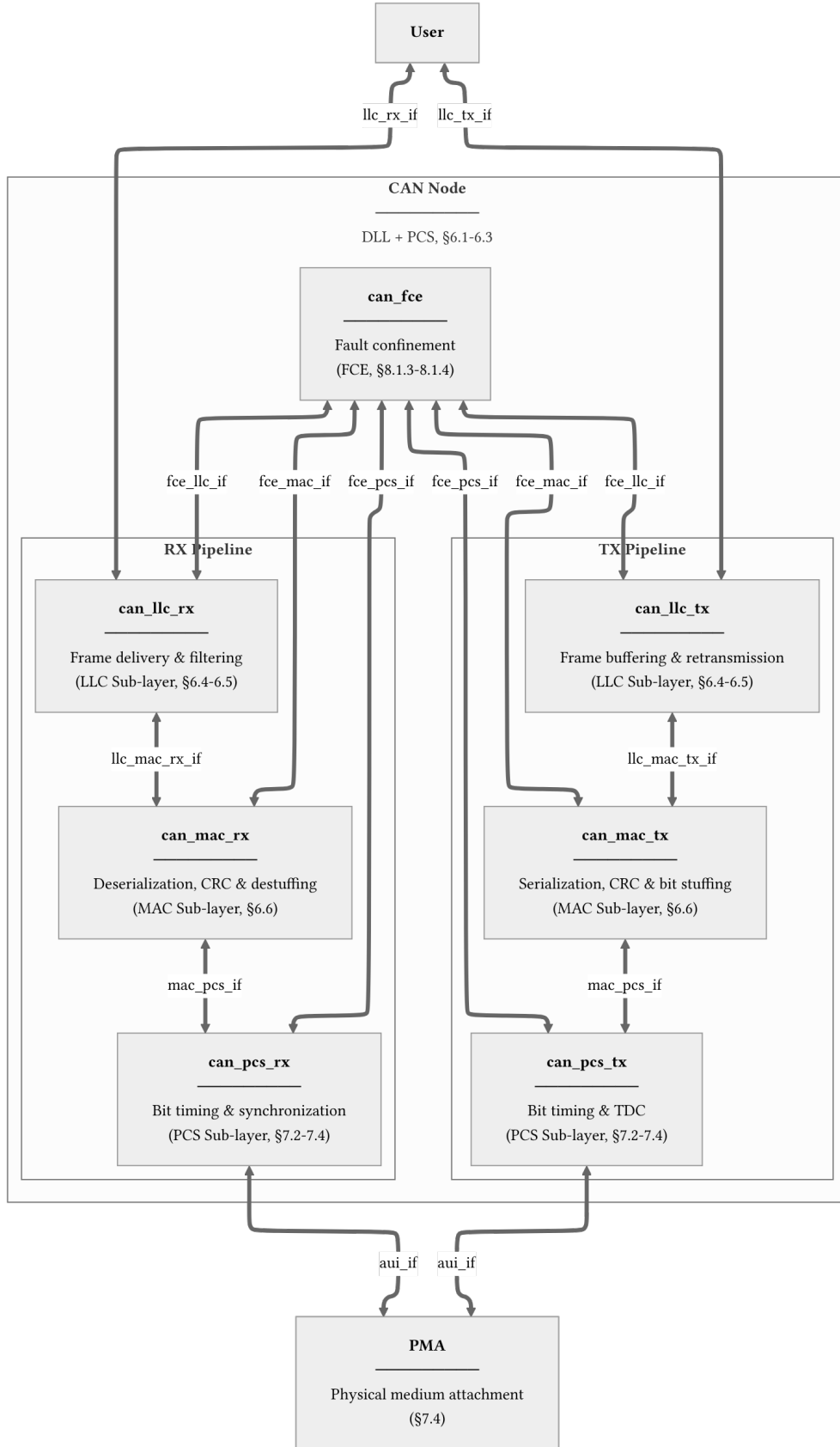


Figure 1: CAN node decomposition across LLC, MAC, PCS, FCE, and PMA boundaries. Interface definitions are provided for `llc_tx_if` (Table 2), `llc_rx_if` (Table 3), `llc_mac_tx_if` (Table 4), `llc_mac_rx_if` (Table 5), `mac_pcs_if` (Table 6), `auif` (Table 7), `fce_llc_if` (Table 8), `fce_mac_if` (Table 9), and `fce_pcs_if` (Table 10).

expands the data field to 64 bytes, and repurposes reserved bits in the last byte for BRS and ESI flags.

The FMT encodes the supported frame formats as ‘000’ = CB, ‘100’ = CE, ‘010’ = FB, ‘110’ = FE. Accordingly, for the frame content to be self-consistent the IDE bit must be set to 0 for FMT = CB/FB and 1 for FMT = CE/FE. The revised format is designed to be backward compatible. An implementation that only supports Classic frames can simply ignore the FMT bits and treat all frames as Classic, while an implementation that supports FD can use the FMT field and the additional control bits (BRS and ESI) to distinguish frame types without affecting the existing ID and data field structure.

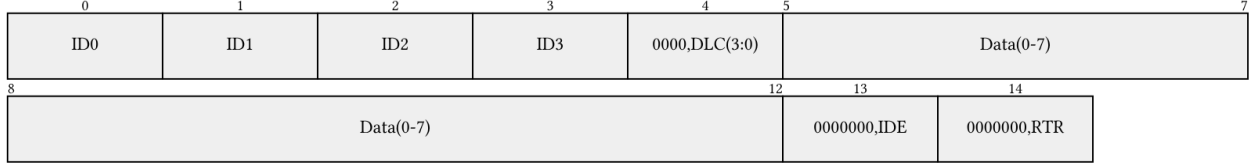


Figure 2: Current LLC frame format (15 bytes). ID byte encoding is defined in Table 1.

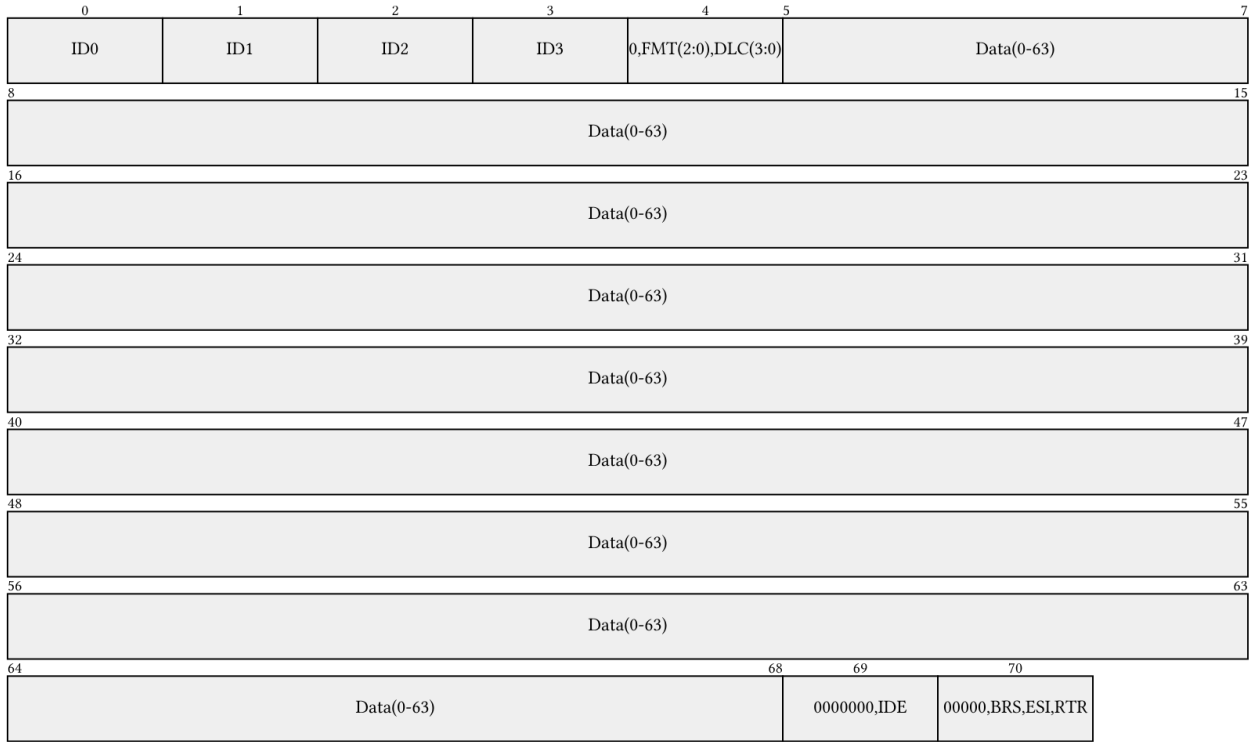


Figure 3: Revised LLC frame format with FD support, shown at maximum length (71 bytes). The FMT field selects frame type; BRS and ESI repurpose reserved bits in the final byte. ID byte encoding is defined in Table 1.

Table 1: ID byte encoding for the LLC frame formats shown in Figure 2 and Figure 3.

Format	Byte ID0	Byte ID1	Byte ID2	Byte ID3
Basic	‘00000000’	‘00000000’	‘00000’ & ‘ID(10-8)’	‘ID(7-0)’
Extended	‘000’ & ‘ID(28:24)’	‘ID(23-16)’	‘ID(15-8)’	‘ID(7-0)’

1.3 Interface Definition Tables

The interface bundles shown in Figure 1 are defined in full below, with each signal mapped to its corresponding ISO 11898-1 service primitive, [1]. This normative anchoring provides a direct traceability path from protocol clauses to VHDL ports. The types used in these interfaces are defined in Section 1.4.

All module interfaces use `std_logic` and `std_logic_vector` exclusively, as mandated for synthesis compatibility. Protocol semantics such as polarity, frame format, and transfer status are encoded as named `std_logic/std_logic_vector` constants rather than enumeration types. The enumeration types and composite record types described in Section 1.4 are used only internally within the MAC layer and on simulation-only debug ports. The LLC-User interface (`llc_tx_if`, `llc_rx_if`) is implemented as an Avalon-ST byte stream to maintain backward compatibility with the existing CAN bus controller, which uses the same `valid/ready/startofpacket` handshake convention.

Table 2: Interface definition for `llc_tx_if`. Implements `L_Data.Request` as an Avalon-ST byte stream. `L_Data.AbortRequest` is included for complete ISO service coverage but is not yet implemented.

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
6.4.5.5.2	<code>L_Data.Request</code>	LLC Frame, Handle	Submit LLC frame for transmission	User -> LLC	<code>llc_tx_if.data (byte_t)</code> , <code>llc_tx_if.valid (std_logic)</code> , <code>llc_tx_if.sop (std_logic)</code>	valid high with byte on data; sop asserted on first byte
6.4.5.5.2	<code>L_Data.Request</code>	LLC Frame, Handle	Submit LLC frame for transmission	User -> LLC	<code>llc_tx_if.data (byte_t)</code> , <code>llc_tx_if.valid (std_logic)</code>	valid high; sop and eop deasserted on intermediate bytes
6.4.5.5.2	<code>L_Data.Request</code>	LLC Frame, Handle	Submit LLC frame for transmission	User -> LLC	<code>llc_tx_if.data (byte_t)</code> , <code>llc_tx_if.valid (std_logic)</code> , <code>llc_tx_if.eop (std_logic)</code>	valid high with byte on data; eop asserted on last byte
6.4.5.5.2	<code>L_Data.Request</code>		Flow control	LLC -> User	<code>llc_tx_if.ready (std_logic)</code>	Asserted by LLC when able to consume next byte
6.4.5.5.3	<code>L_Data.AbortRequest</code>	Handle	Abort pending frame transfer	User -> LLC	<code>llc_tx_if.abort_request (std_logic)</code>	Pulse when user wants to cancel an in-progress transfer

Table 3: Interface definition for `llc_rx_if`. Implements `L_Data.Indication` as an Avalon-ST byte stream. Timestamp is included for complete ISO service coverage but is not yet implemented.

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
6.4.5.5.5	<code>L_Data.Indication</code>	LLC Frame, Timestamp	Deliver received LLC frame to user	LLC -> User	<code>llc_rx_if.data (byte_t)</code> , <code>llc_rx_if.valid (std_logic)</code> , <code>llc_rx_if.sop (std_logic)</code>	valid high with byte on data; sop asserted on first byte

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
6.4.5.5.5	L_Data.Indication	LLC Frame, Timestamp	Deliver received LLC frame to user	LLC -> User	llc_rx_if.data (byte_t), llc_rx_if.valid (std_logic)	valid high; sop and eop deasserted on intermediate bytes
6.4.5.5.5	L_Data.Indication	LLC Frame, Timestamp	Deliver received LLC frame to user	LLC -> User	llc_rx_if.data (byte_t), llc_rx_if.valid (std_logic), llc_rx_if.eop (std_logic)	valid high with byte on data; eop asserted on last byte
6.4.5.5.5	L_Data.Indication		Flow control	User -> LLC	llc_rx_if.ready (std_logic)	Asserted by user when able to consume next byte
6.4.5.5.4	L_Data.Confirm	Transfer_Status, Timestamp, Handle	Report outcome of prior L_Data.Request	LLC -> User	llc_rx_if.transfer_status (std_logic_vector(2:0))	Issued on frame completion, loss, or error

Table 4: Interface definition for llc_mac_tx_if. Frame length is self-describing from the DLC config bytes; the MAC serializer does not consume eop.

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
6.3, 6.6.4.2	DLL SDU	LLC Frame	Transfer LLC frame to MAC for serialization	LLC -> MAC	llc_mac_tx_if.data (byte_t), llc_mac_tx_if.valid (std_logic), llc_mac_tx_if.sop (std_logic)	valid high with byte on data; sop marks first byte of new frame
6.3, 6.6.4.2	DLL SDU		Flow control	MAC -> LLC	llc_mac_tx_if.ready (std_logic)	Asserted by MAC serializer when able to consume next byte
6.6.4.2	Interface control information	Transfer_Status	Report frame transmission outcome	MAC -> LLC	llc_mac_tx_if.transfer_status (std_logic_vector(2:0))	Updated by MAC on completion, loss, or error

Table 5: Interface definition for `llc_mac_rx_if`. Carries the reconstructed LLC frame from `can_mac_rx` to `can_llc_rx` (Section 1.6.2). Full decomposition of RX sub-module interfaces is deferred to future work.

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
6.6.4.3, 6.6.9	DLL SDU	Reconstructed LLC Frame	Transfer reconstructed LLC frame to LLC	MAC -> LLC		Issued when MAC has reconstructed a frame from the received bitstream
6.6.3	Time reference notification	SOF/frame-valid indication	Provide timing reference for timestamping	MAC -> LLC		Generated for transmitted/received DF/RF

Table 6: Interface definition for `mac_pcs_if`. The `D_Transmit` status is signalled via a dedicated `use_data_rate` flag rather than encoding it in a semantic bit name. The PCS switches between nominal and data-phase bit timing based on this flag, keeping the interface to plain `std_logic` signals.

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
7.2.1, 7.2.2	PCS_Data.Request	Output_Unit	Present next bit for transmission	MAC -> PCS	<code>mac_pcs_if.polarity (std_logic)</code> , <code>mac_pcs_if.valid (std_logic)</code>	MAC holds polarity stable; PCS samples at each bit boundary autonomously
7.2.1, 7.2.5	PCS_Status.Transmitter	D_Transmit	Indicate FD data-phase interval to PCS	MAC -> PCS	<code>mac_pcs_if.use_data_rate (std_logic)</code> , <code>mac_pcs_if.start_tdc (std_logic)</code>	<code>use_data_rate</code> asserted during FD data phase; <code>start_tdc</code> pulsed at BRS to begin delay measurement
7.2.1, 7.2.3	PCS_Data.Indicate	Input_Unit	Indicate bus polarity at sample point	PCS -> MAC	<code>mac_pcs_if.bus_polarity (std_logic)</code> , <code>mac_pcs_if.sp (std_logic)</code> , <code>mac_pcs_if.ssp (std_logic)</code> , <code>mac_pcs_if.fifo_index (t_fifo_index_vec)</code>	<code>sp</code> pulses once per nominal sample point; <code>ssp</code> pulses once per secondary sample point for TDC

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
7.2.1, 7.2.6	PCS_Status.Receiver	D_Receive	Indicate FD data-phase interval to PCS	MAC -> PCS	not yet implemented	Asserted during FD data-phase reception

Table 7: Interface definition for `auif`. All signals are `std_logic`, as this interface crosses from the ISO protocol domain into the physical medium.

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
7.4.2.1	output symbol	Dominant/recessive symbol	Drive physical output symbol	PCS -> PMA	<code>auif.tx(std_logic)</code>	Updated on each <code>Output_Unit</code> request
7.4.2.2, 8.1.3.4	bus_off symbol	Bus-off control	Switch node off bus	PCS -> PMA	<code>auif.bus_off(std_logic)</code>	On <code>Bus_off_request</code> from FCE
7.4.2.3, 8.1.3.4	bus_off_release symbol	Bus-off release control	Release node from bus-off	PCS -> PMA	<code>auif.bus_off_release(std_logic)</code>	On <code>Bus_off_release_request</code> from FCE
7.4.3	input symbol	Dominant/recessive symbol	Indicate physical input symbol	PMA -> PCS	<code>auif.rx(std_logic)</code>	Continuously driven by PMA

Table 8: Interface definition for `fce_llc_if`. Carries bus-off status and mode-request handshake between the FCE and LLC layers [1, Sec. 8.1.3.2].

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
8.1.3.2	Normal_mode_request	Mode request	Request reset to normal mode	LLC -> FCE	<code>fce_llc_if.normal_mode_request(std_logic)</code>	Issued on startup/restart

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
8.1.3.2	Normal_mode_response	Mode response	Acknowledge normal-mode request	FCE -> LLC	fce_llc_if.normal_mode_response (std_logic)	Returned after FCE processing
8.1.3.2	Bus_off	Bus-off status	Indicate node is bus-off	FCE -> LLC	fce_llc_if.bus_off (std_logic)	Asserted on bus-off transition

Table 9: Interface definition for fce_mac_if. The MAC reports error events and frame outcomes to the FCE; the FCE returns the current error-active/passive state to the MAC [1, Sec. 8.1.3.3].

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
8.1.3.3	Transmit/receive	Transfer mode context	Report current TX/RX context	MAC -> FCE	fce_mac_if.transmitting (std_logic)	Updated with MAC transfer context
8.1.3.3	Error	Error event	Report detected protocol error	MAC -> FCE	fce_mac_if.error (std_logic)	Pulse on bit/stuff/CRC/form/ACK error
8.1.3.3	Primary_error	Primary error event	Report primary error condition	MAC -> FCE	fce_mac_if.primary_error (std_logic)	Pulse on primary error condition
8.1.3.3	Error/overload flag	EF/OF state	Report EF/OF transmission state	MAC -> FCE	fce_mac_if.sending_error_overload (std_logic)	Asserted during EF/OF transmission
8.1.3.3	Counters_unchanged	Counter-update qualifier	Qualify counter exception path	MAC -> FCE	fce_mac_if.counters_unchanged (std_logic)	Asserted on rule-c exception cases
8.1.3.3	Error_delimiter_too_late	Late delimiter event	Report late error-delimiter condition	MAC -> FCE	fce_mac_if.error_delimiter_too_late (std_logic)	Asserted on late delimiter condition
8.1.3.3	Successful_transfer	Transfer completion event	Report successful TX/RX completion	MAC -> FCE	fce_mac_if.successful_transfer (std_logic)	Pulse on successful frame transfer

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
8.1.3.3	Error_passive_response	State response	Report entry into error-passive state	MAC -> FCE	fce_mac_if.error_passive_respons (std_logic)	On state transition completion
8.1.3.3	Error_active_response	State response	Report return to error-active state	MAC -> FCE	fce_mac_if.error_active_response (std_logic)	On state transition completion
8.1.3.3	Error_passive_request	State request	Request MAC enter error-passive state	FCE -> MAC	fce_mac_if.error_passive_request (std_logic)	On TEC/REC threshold crossing
8.1.3.3	Error_active_request	State request	Request MAC return to error-active state	FCE -> MAC	fce_mac_if.error_active_request (std_logic)	On TEC/REC recovery

Table 10: Interface definition for fce_pcs_if. Carries bus-off and bus-off-release request/response handshakes between the FCE and PCS layers [1, Sec. 8.1.3.4].

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
8.1.3.4	Bus_off_request	Bus-off request	Request node switch-off from bus	FCE -> PCS	fce_pcs_if.bus_off_request (std_logic)	On bus-off transition condition
8.1.3.4	Bus_off_release_request	Bus-off release request	Request node re-enable from bus-off	FCE -> PCS	fce_pcs_if.bus_off_release_requ (std_logic)	On restart/reintegration
8.1.3.4	Bus_off_response	Bus-off response	Acknowledge bus-off request	PCS -> FCE	fce_pcs_if.bus_off_response (std_logic)	Returned after bus-off action
8.1.3.4	Bus_off_release_response	Bus-off release response	Acknowledge bus-off-release request	PCS -> FCE	fce_pcs_if.bus_off_release_resp (std_logic)	Returned after release action

1.4 Protocol-Driven Type System

The type system encodes protocol semantics into named constants and internal record types, as shown in Figure 4. At module boundaries, all signals use `std_logic` and `std_logic_vector` exclusively - protocol concepts such as polarity, frame format, and transfer status are represented as named constant groups. Internally, enumeration types (`t_mac_frame_bit_name`, `t_monitor_event_tx`) and composite record types (`t_bit`, `t_mac_frame_bit`, `t_frame_params`) make protocol semantics visible in simulation waveforms and debug ports.

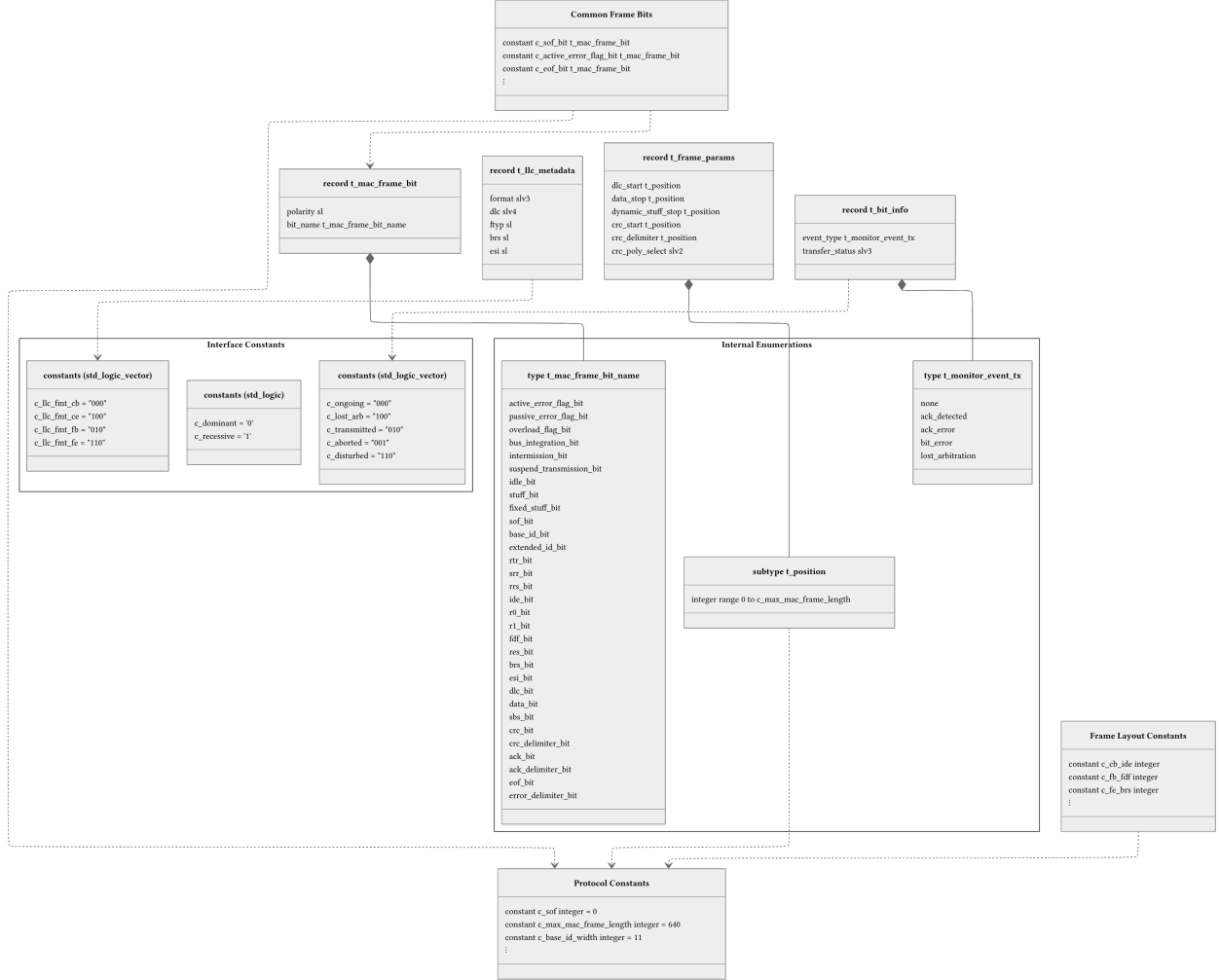


Figure 4: Type and constant hierarchy. The Interface Constants namespace groups the `std_logic` and `std_logic_vector` constants that encode protocol semantics at module boundaries. The Internal Enumerations namespace contains enumeration types used only within the MAC layer and on simulation-only debug ports. Compound record types compose these primitives: `t_mac_frame_bit` carries semantic context by combining a polarity with a protocol bit name, so each transmitted bit remains identifiable throughout the design. `t_llc_metadata` captures the five LLC config byte fields - named to match their ISO 11898-1 definitions - passed from the serializer to the FSM. `t_frame_params` aggregates derived field boundary positions and CRC polynomial selection, computed once per frame by the FSM (Section 1.6.1.2). `t_bit_info` supports bus monitoring for error detection and arbitration loss (Section 1.6.1.2). Constant groups derive from the root protocol constants.

1.5 LLC Sub-layer

Responsible for frame buffering and retransmission management. It provides an Avalon-ST interface to the user application and communicates with the FCE to handle retransmission limits and error status reporting.

1.5.1 `can_llc_tx`

`can_llc_tx` buffers an incoming LLC frame from the user, streams it byte-by-byte to the MAC serializer, and manages retransmission on bus disturbance up to the ISO-mandated limit [1, Sec. 6.4.5] and 6.5.

1.5.2 `can_llc_rx`

`can_llc_rx` receives a reconstructed LLC frame from the MAC layer, applies acceptance filtering, and delivers accepted frames to the user over the `llc_rx_if` Avalon-ST stream [1, Sec. 6.4.5].

1.6 MAC Sub-layer

The MAC sub-layer is the core of the protocol logic, responsible for bit serialization, CRC generation, bit stuffing, and frame-level error detection. It coordinates closely with the FCE (Section ??) for error counter management and node-state transitions (Error Active/Passive/Bus Off), and with the PCS (Section 1.7) for sample-point-driven bit output.

The earlier CAN bus controller concentrated TX, RX, MAC, and FCE logic in a single monolithic FSM (`can_fsm`), with the sub-functions - serialization (`can_ast_to_serial`), bit stuffing (`can_stuff_bit_gen`), and CRC (`gen_crc`) - implemented in satellite modules driven directly by it. PCS timing logic was implemented in `can_node_clock`, which fed sample-point and transmit pulses into `can_fsm` - a strategy retained in the current design.

The current design restructures these modules around the ISO 11898-1 [1] layer boundaries. On the TX side the mapping is direct: `can_ast_to_serial` becomes `can_mac_ser_tx` (Section 1.6.1.3), `can_stuff_bit_gen` becomes `can_mac_bs_tx` (Section 1.6.1.4), `gen_crc` becomes `can_mac_crc_tx` (Section 1.6.1.5), and `can_node_clock` becomes `can_pcs_tx` (Section 1.7.1). The RX-side counterparts - deserialization (`can_serial_to_ast`), bit destuffing, and CRC checking - will map to submodules within `can_mac_rx` (Section 1.6.2). Fault confinement, previously embedded in `can_fsm`, is separated into its own layer.

1.6.1 `can_mac_tx`

The `can_mac_tx` (Figure 5) is composed of four main components:

1. **`can_mac_ser_tx`**: Converts LLC bytes into a serial bit stream, extracts LLC metadata from config bytes
2. **`can_mac_fsm_tx`**: Coordinates frame transmission, computes frame parameters from LLC metadata, controls all submodules
3. **`can_mac_bs_tx`**: Implements CAN/CAN-FD bit stuffing rules
4. **`can_mac_crc_tx`**: Generates CRC-15/CRC-17/CRC-21 based on frame type

1.6.1.1 `can_mac_tx` Internal Interfaces The interfaces between MAC sub-components are implementation-defined and carry no direct ISO service primitive mapping. Table 11, Table 12, and Table 13 define each bidirectional bundle; the Direction column identifies the driving component for each field.

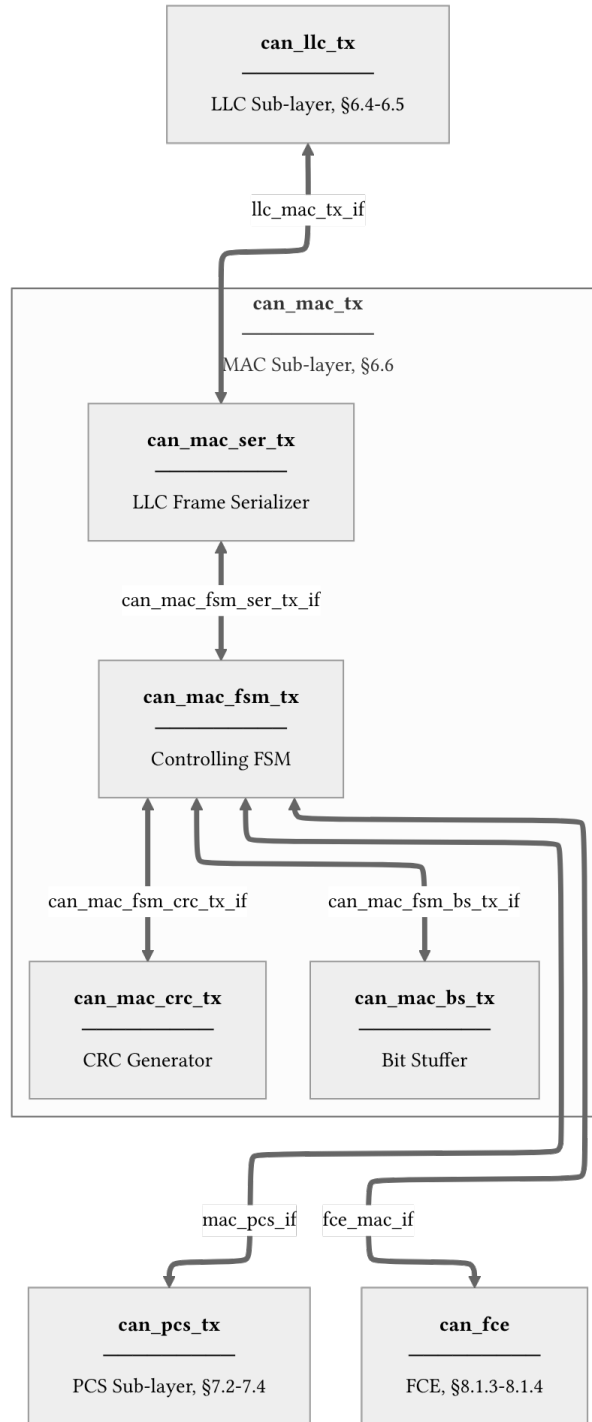


Figure 5: **can_mac_tx** architecture showing internal component interconnections and external interfaces. Internal interface definitions are provided for **can_mac_fsm_ser_tx_if** (Table 11), **can_mac_fsm_bs_tx_if** (Table 12), and **can_mac_fsm_crc_tx_if** (Table 13).

Table 11: interface definition for `can_mac_fsm_ser_tx_if`, connecting `can_mac_ser_tx` and `can_mac_fsm_tx` (see Figure 5).

field	type	direction	description
<code>data</code>	<code>std_logic</code>	<code>ser → fsm</code>	current bit polarity; fsm reads this when <code>valid</code> is asserted
<code>valid</code>	<code>std_logic</code>	<code>ser → fsm</code>	asserted while a bit is available for the fsm to consume
<code>llc_metadata</code>	<code>t_llc_metadata</code>	<code>ser → fsm</code>	LLC config byte fields (format, DLC, flags) extracted by the serializer; valid for the lifetime of the frame
<code>ready</code>	<code>std_logic</code>	<code>fsm → ser</code>	pulsed when the fsm has consumed the current bit; advances the serializer to the next bit
<code>transfer_status</code>	<code>std_logic_vector(2:0)</code>	<code>fsm → ser</code>	frame outcome; any non- <code>c_ongoing</code> value terminates serialization

Table 12: interface definition for `can_mac_fsm_bs_tx_if`, connecting `can_mac_fsm_tx` and `can_mac_bs_tx` (see Figure 5).

field	type	direction	description
<code>data</code>	<code>std_logic</code>	<code>fsm → bs</code>	bit polarity fed into the bit stuffer
<code>valid</code>	<code>std_logic</code>	<code>fsm → bs</code>	pulsed when a new bit is presented to the stuffer
<code>start</code>	<code>std_logic</code>	<code>fsm → bs</code>	pulsed at frame start to reinitialize the stuffer state
<code>data</code>	<code>std_logic</code>	<code>bs → fsm</code>	polarity of the required stuff bit
<code>valid</code>	<code>std_logic</code>	<code>bs → fsm</code>	asserted when a stuff bit insertion is required
<code>sbc</code>	<code>std_logic_vector(3:0)</code>	<code>bs → fsm</code>	gray-coded stuff bit count with parity, used in the fd fixed-stuffing region

Table 13: interface definition for `can_mac_fsm_crc_tx_if`, connecting `can_mac_fsm_tx` and `can_mac_crc_tx` (see Figure 5).

field	type	direction	description
<code>crc_poly_select</code>	<code>std_logic_vector(1:0)</code>	<code>fsm → crc</code>	selects the active crc polynomial: crc-15, crc-17, or crc-21
<code>valid</code>	<code>std_logic</code>	<code>fsm → crc</code>	pulsed when data should be included in the crc accumulation
<code>data</code>	<code>std_logic</code>	<code>fsm → crc</code>	bit value to feed into the crc engine
<code>start</code>	<code>std_logic</code>	<code>fsm → crc</code>	pulsed to reinitialize the crc engine
<code>crc</code>	<code>t_crc_vector</code>	<code>crc → fsm</code>	current crc register value; fsm reads this when serializing the crc field

1.6.1.2 can_mac_fsm_tx `can_mac_fsm_tx` (Figure 6) orchestrates frame transmission, coordinating the serializer (Section 1.6.1.3), bit stuffer (Section 1.6.1.4), and CRC engine (Section 1.6.1.5). When a frame request arrives in `s_bus_idle`, the FSM transitions to `s_transmitting_frame` and computes all format-dependent field boundaries up front:

```
get_frame_params(metadata : t_llc_metadata) -> t_frame_params
```

This call derives `dlc_start`, `data_stop`, `dynamic_stuff_stop`, `crc_start`, `crc_delimiter`, and `crc_poly_select` from the LLC metadata, so the per-bit loop never recalculates frame geometry.

On each sample-point strobe from `can_pcs_tx` (Section 1.7.1), the `s_transmitting_frame` state executes the following sequence:

1. **Monitor the bus.** The sampled bus polarity is compared against the previously transmitted bit to detect errors, ACK, and arbitration loss. In the CAN-FD data phase, any pending SSP observation from the previous bit period is evaluated first [1, Sec. 7.3.4].

```
get_bit_info(bit_name           : t_mac_frame_bit_name,
             polarity_history    : t_tdc_polarity_history,
             tdc_delay           : integer,
             monitored_bit_polarity : std_logic,
             metadata            : t_llc_metadata) -> t_bit_info
```

The returned `t_bit_info` carries a `t_monitor_event_tx` (none, `ack_detected`, `ack_error`, `bit_error`, or `lost_arbitration`) and a `transfer_status` encoding. A detected error triggers a transition to the appropriate error flag state based on the FCE fault confinement status [1, Secs. 8.1.3–8.1.4]. Arbitration loss causes the FSM to stop driving and return to `s_intermission`.

2. **Determine the next bit.** If the bit stuffer has a pending stuff bit, that takes priority. Otherwise, the next protocol bit is computed from the current `bit_count` and the cached `t_frame_params`:

```
get_mac_frame_bit(bit_count      : t_position,
```

```

ser_data      : std_logic,
metadata      : t_llc_metadata,
frame_params  : t_frame_params,
previous_polarity : std_logic,
sbc           : t_sbc,
crc           : t_crc_vector) -> t_mac_frame_bit

```

The returned `t_mac_frame_bit` pairs a polarity with a `t_mac_frame_bit_name`, identifying both the value to drive and the protocol field it belongs to.

3. **Feed the CRC engine and bit stuffer, increment `bit_count`.** The output polarity is forwarded to the CRC engine (for bits within the CRC-protected region) and to the bit stuffer (for bits within the dynamic stuffing region, bounded by `dynamic_stuff_stop` in `t_frame_params`). The FSM then advances `bit_count` for the next sample point.
4. **Present the bit at the PCS interface.** The resolved polarity is driven onto the `t_can_mac_pcs_tx_if_m2s` interface together with the appropriate control signals: `use_data_rate` is asserted during the CAN-FD data phase to switch the PCS to the faster bit timing, and `start_tdc` is pulsed at the FDF bit to initiate transmitter delay compensation [1, Sec. 7.3.4].

1.6.1.3 `can_mac_ser_tx` `can_mac_ser_tx` converts the LLC byte stream into a serial polarity bit stream for the MAC FSM. Its four-state FSM (Figure 7) manages the two-byte configuration handshake, byte fetching, and bit-by-bit serialization. The serializer extracts LLC metadata (format, DLC, flags) from the two config bytes and passes it to the FSM via `t_llc_metadata`. The FSM then computes `t_frame_params` internally from this metadata.

1.6.1.4 `can_mac_bs_tx` `can_mac_bs_tx` performs dynamic bit stuffing for both CAN Classic and CAN-FD frames [1, Sec. 10.6]. It monitors the polarity stream from the FSM (Section 1.6.1.2) and inserts an inverse-polarity stuff bit after every five consecutive identical bits. It also maintains a Gray-coded Stuff Bit Count (SBC) with parity for the CAN-FD CRC field. The data path diagram is shown in Figure 8.

1.6.1.5 `can_mac_crc_tx` `can_mac_crc_tx` provides CRC generation for both CAN Classic and CAN-FD frames. CAN Classic frames use CRC-15, while CAN-FD frames use CRC-17 (data payloads up to 16 bytes) or CRC-21 (data payloads above 16 bytes) [1, Sec. 10.4.2.6]. The FSM selects the appropriate polynomial via the `crc_poly_select` field (Table 13). The data path diagram is shown in Figure 9.

1.6.2 `can_mac_rx`

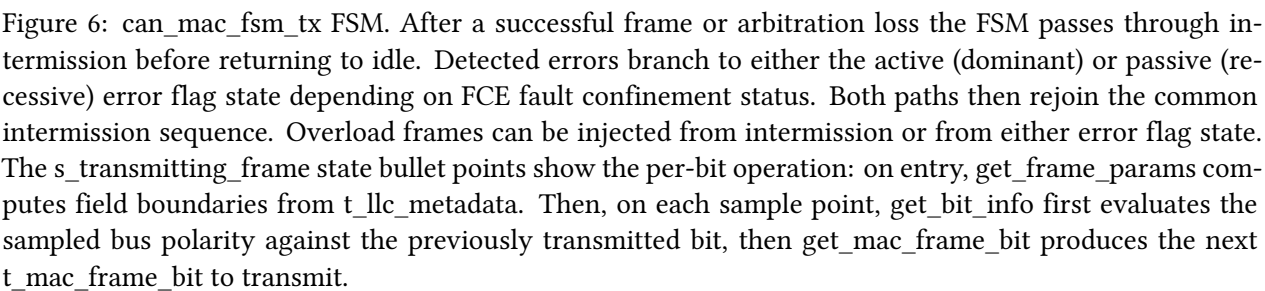
`can_mac_rx` deserializes the bit stream received from the PCS, performs bit destuffing and CRC verification, and reconstructs the LLC frame for delivery to `can_llc_rx` [1, Sec. 6.6].

1.7 PCS Sub-layer

Handles bit timing and synchronization. It generates the sample point (SP) and secondary sample point (SSP) strobes. It provides bit-level monitoring data to the FCE to detect synchronization and timing errors.

1.7.1 `can_pcs_tx`

`can_pcs_tx` handles bit timing and Transmitter Delay Compensation (TDC) for the TX path [1, Secs. 7.2–7.3]. It generates the sample point (SP) strobe used by the MAC FSM to advance frame state, and - when TDC



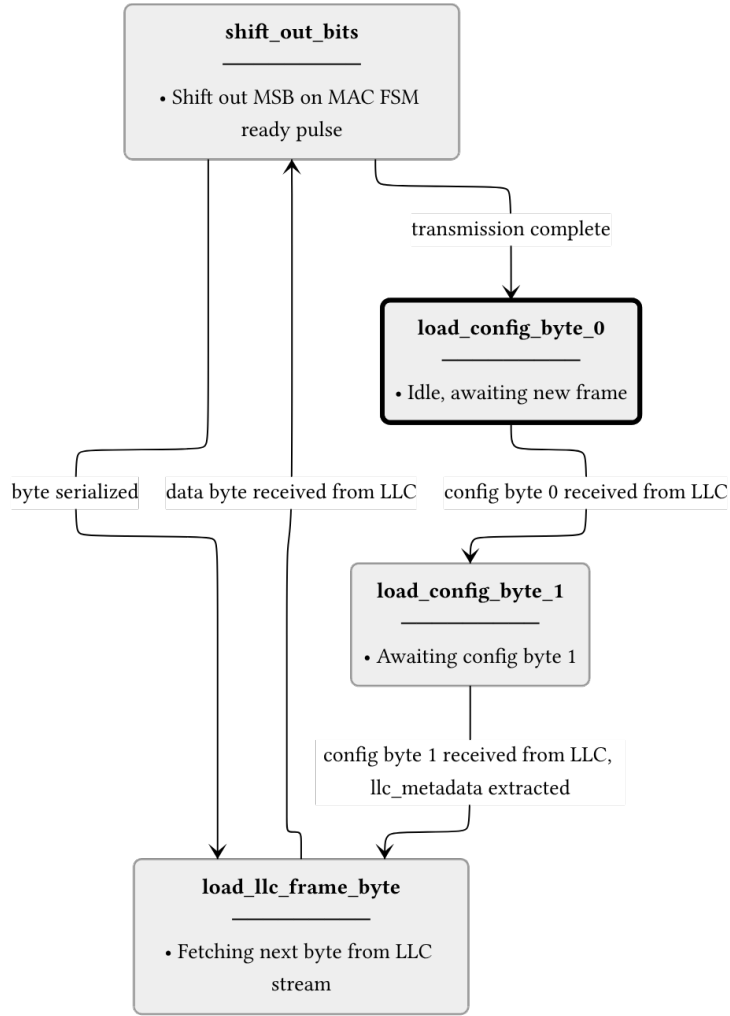


Figure 7: can_mac_ser_tx FSM. The serializer accepts LLC frame bytes over a ready/valid handshake and shifts them out MSB-first to the MAC FSM one bit per ready pulse. LLC metadata is extracted from the two config bytes and registered in t_llc_metadata for use by the FSM throughout the frame.

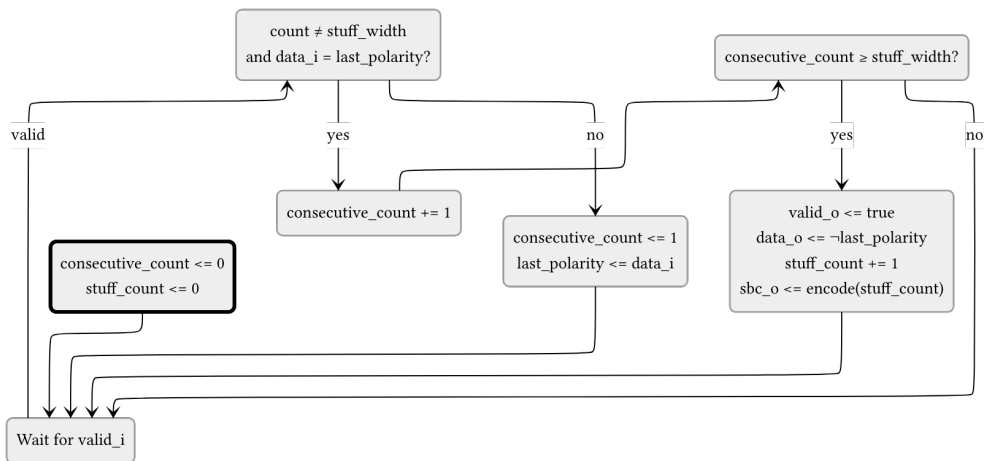


Figure 8: can_mac_bs_tx data path diagram. When valid and data = last_polarity, consecutive_count increments. Otherwise it restarts at 1 with last_polarity updated to data. When consecutive_count reaches stuff_width, valid and inverted data are driven and stuff_count increments. The to_gray() + calc_parity() block encodes stuff_count into the sbc output. The start pulse resets consecutive_count and stuff_count to zero. All outputs are registered.

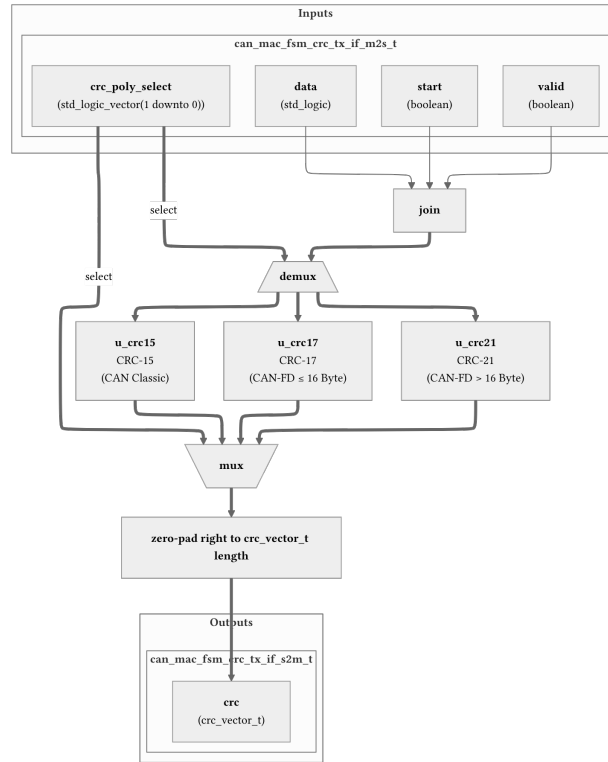


Figure 9: can_mac_crc_tx data path diagram. data, valid, and start are joined and demuxed by crc_poly_select to gate only the selected CRC engine. The mux selects the active engine result, which is zero-padded to the common crc_vector_t width. All outputs are registered.

is configured - a secondary sample point (SSP) strobe for data-phase bit monitoring. The FSM (Figure 10) has four states reflecting the two bit rates and the TDC measurement window. The measuring_delay state determines the Transmitter Delay Compensation Value (TDCV, [1, Sec. 7.3.4]) by observing the TX-to-RX propagation delay, which together with the configured TDCO offset defines the SSP position for subsequent data-phase bits.

1.7.2 can_pcs_rx

can_pcs_rx implements bit timing and synchronization for the RX path. It performs hard and soft synchronization on the incoming bus signal and generates the sample point strobe used by the MAC RX layer to latch each received bit [1, Secs. 7.2–7.3].

[1] International Organization for Standardization, *Road vehicles — controller area network (CAN) — Part 1: Data link layer and physical coding sublayer*. 2024.

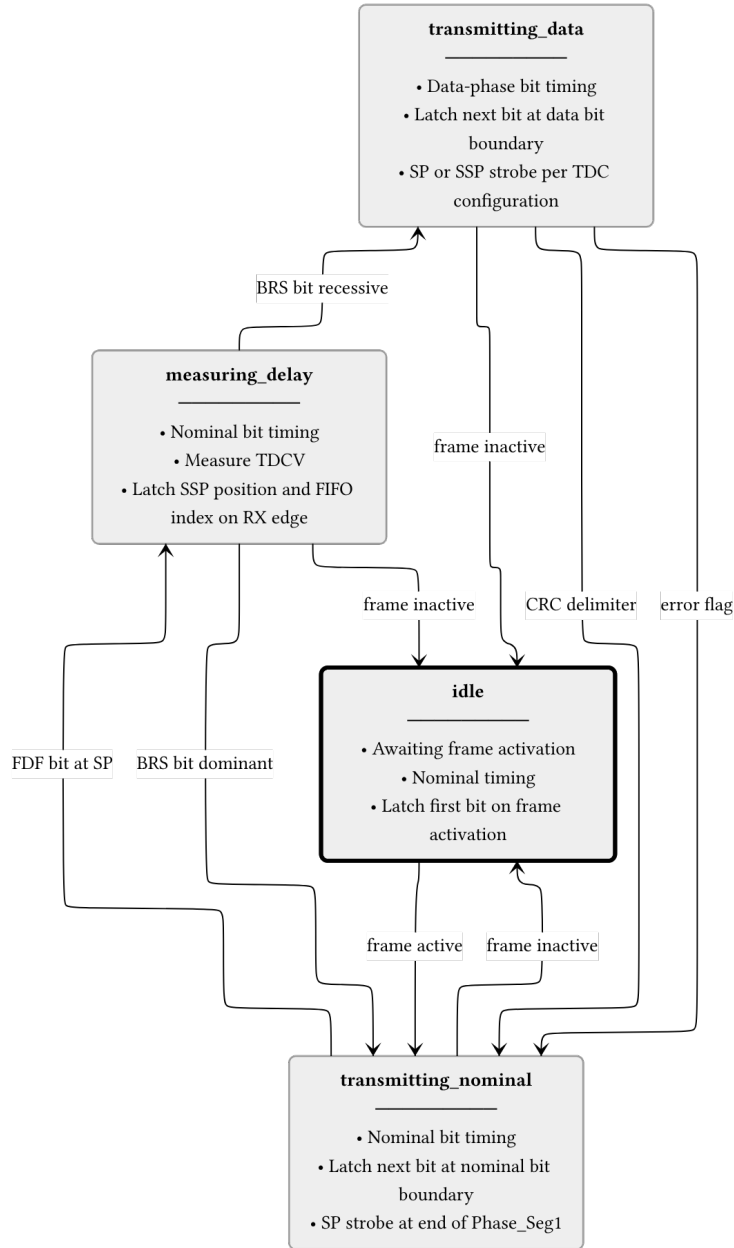


Figure 10: can_pcs_tx FSM. The measuring_delay state is entered on the FDF sample point to measure TDCV (Transmitter Delay Compensation Value, [1], sec. 7.3.4)]. The BRS bit boundary determines whether data-phase timing is used. All non-idle states return to idle when the frame becomes inactive.